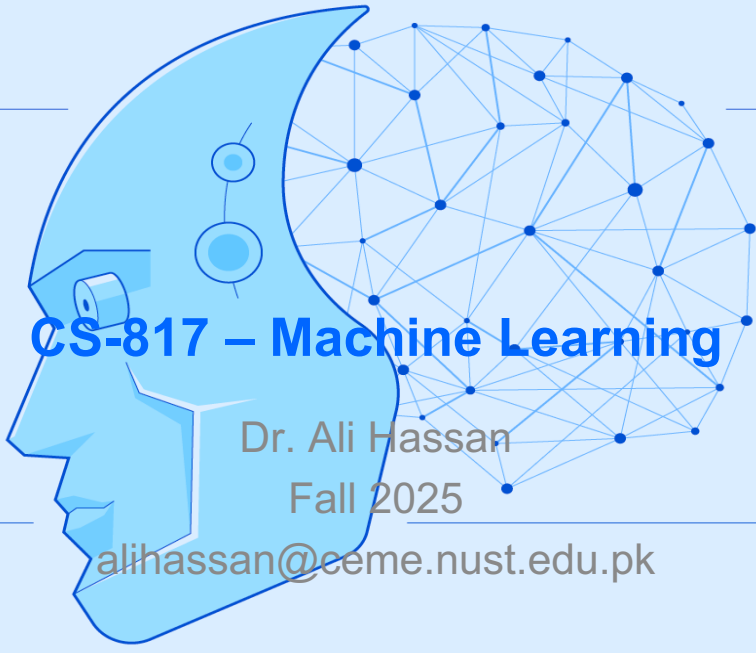




CS-817 – Machine Learning

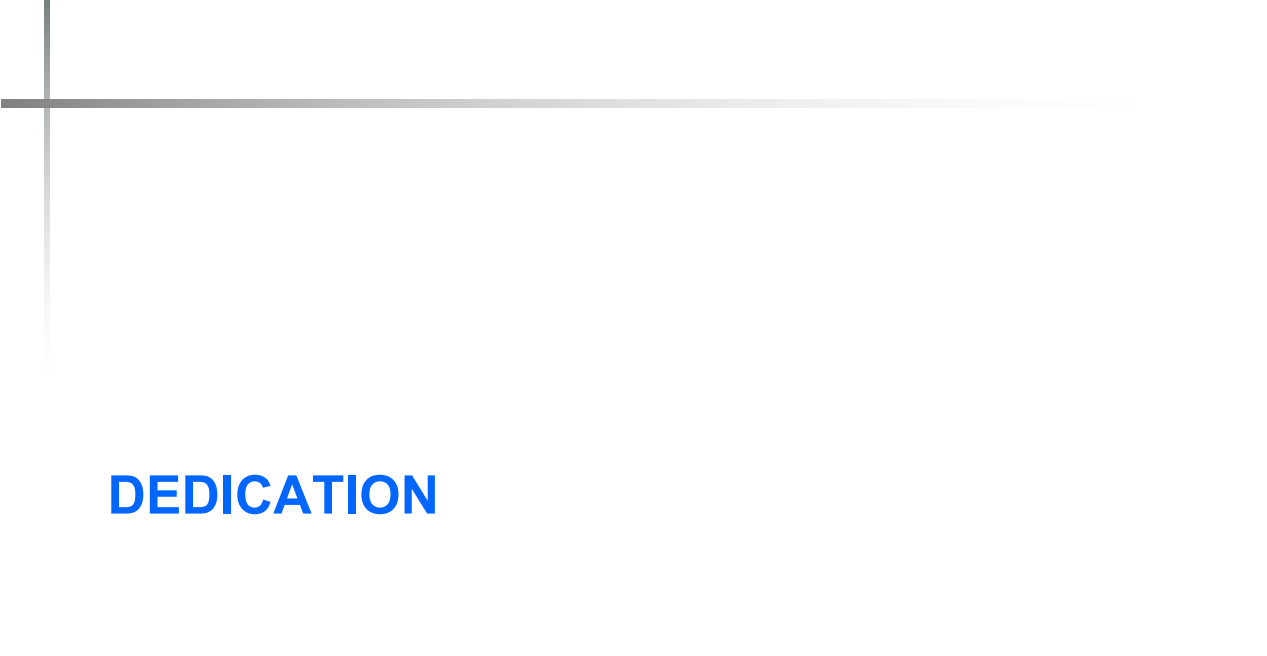
Dr. Ali Hassan
Fall 2025
alihassan@ceme.nust.edu.pk



Dr. Ali Hassan, DC&SE, College of E&ME, NUST

1

1



DEDICATION



Dr. Ali Hassan, DC&SE, College of E&ME, NUST

2

2

Dedication

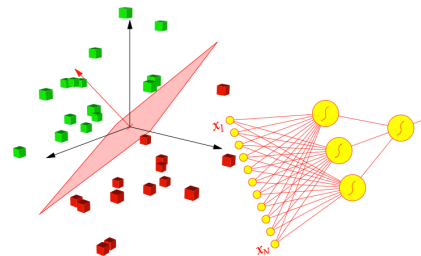


3

3

Announcements

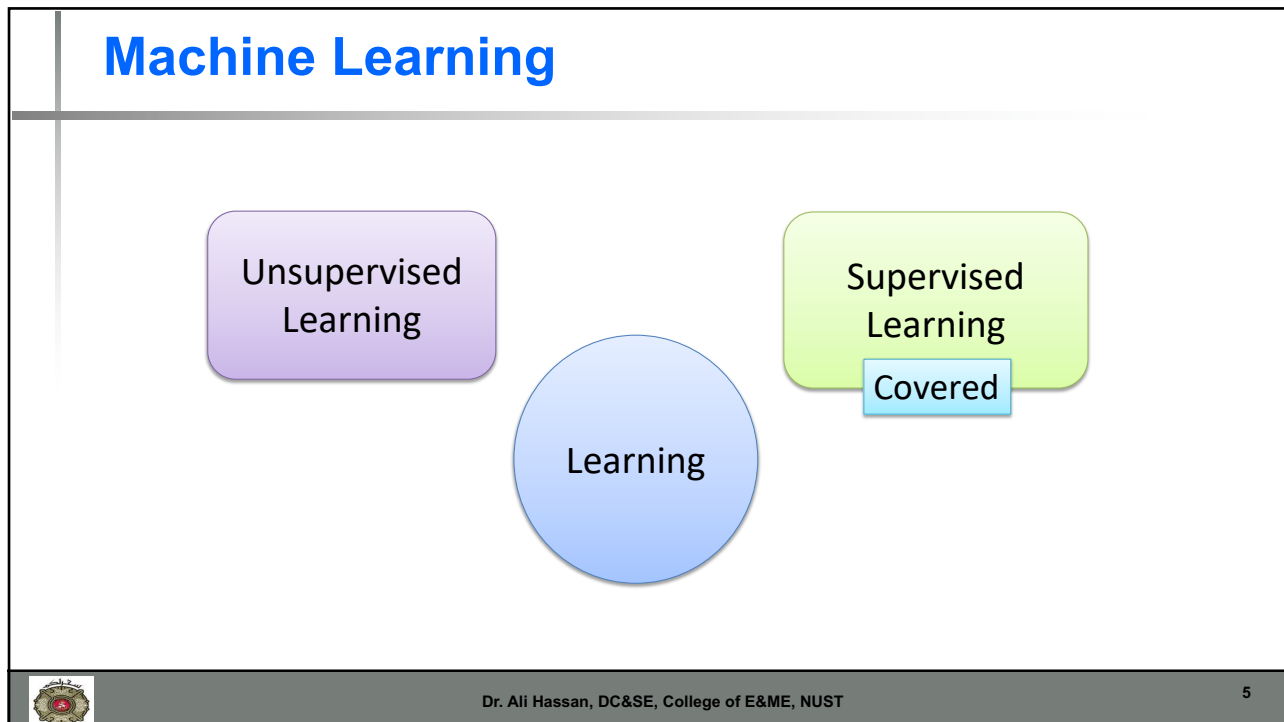
- Assignment 4 on LMS
 - Deadline – 26 November 2025
- Assignment 5
 - Deadline
- Invited Lecture
 - Today on LLMs



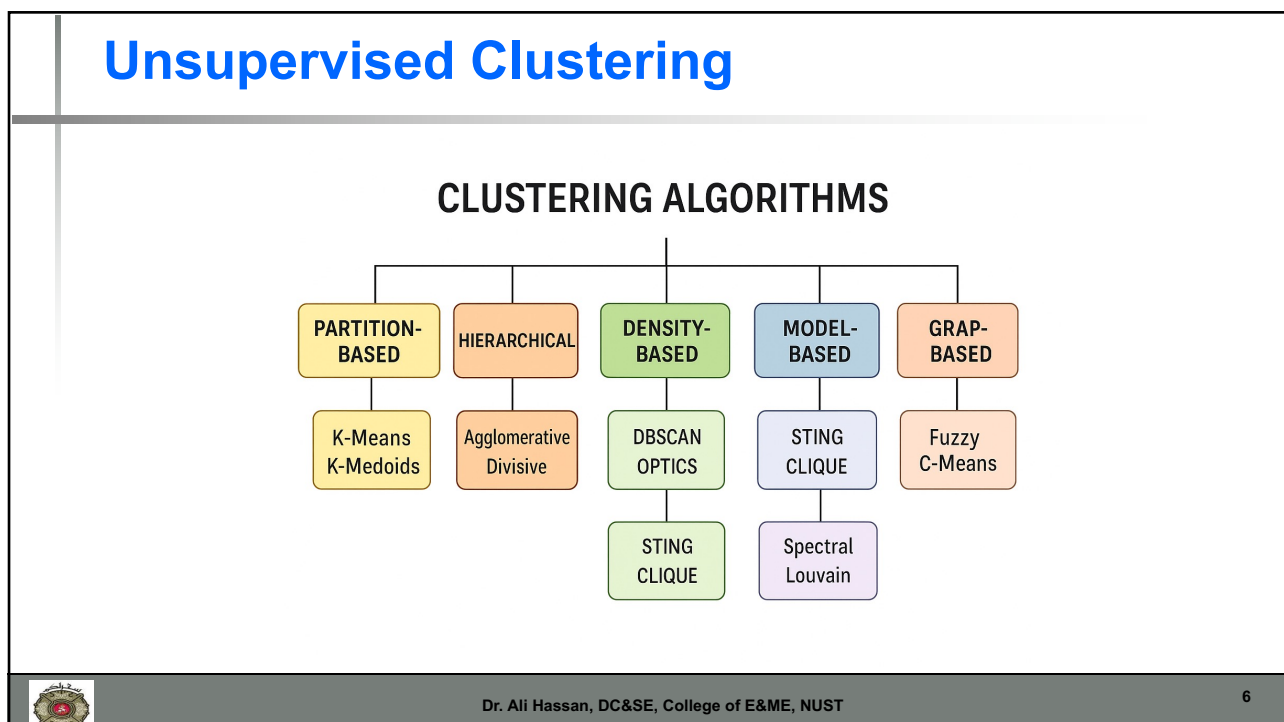
Dr. Ali Hassan, DC&SE, College of E&ME, NUST

4

4



5



6

K-MEANS CLUSTERING



7

7

K-means algorithm

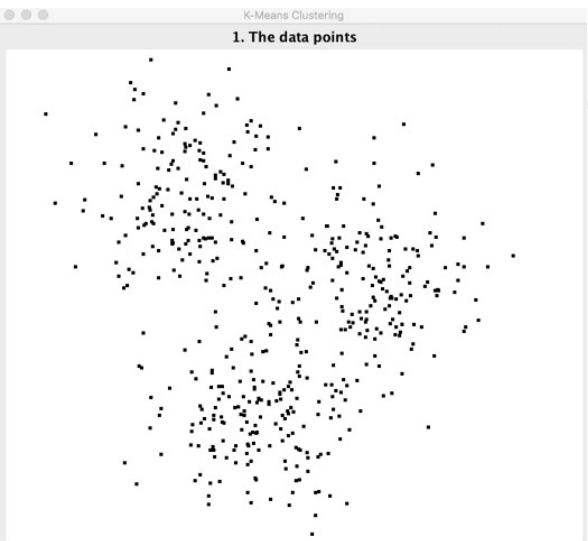
- Randomly initialize K cluster centroids $\mu_1, \mu_2, \dots, \mu_K \in \mathbb{R}^n$
- Repeat {
 - Cluster Assignment {
 - for $i=1$ to m
 - $c^{(i)} :=$ index (1 to K) of cluster centroid closest to $x^{(i)}$
 - Centroid Movement {
 - for $k=1$ to K
 - $\mu_k :=$ mean of points assigned to cluster k



Dr. Ali Hassan, DC&SE, College of E&ME, NUST

8

8



K-Means Clustering

1. The data points

https://youtu.be/nXY6PxAaOk0?si=FYi_dy3NOdOi3hNu

Dr. Ali Hassan, DC&SE, College of E&ME, NUST

9

9



K MEDOIDS

Dr. Ali Hassan, DC&SE, College of E&ME, NUST

10

10

Medoid

K-Medoids is a **clustering algorithm** similar to **K-Means**, but instead of using the *mean (average)* of cluster points as the center, it uses an **actual data point** — called the **medoid** — as the cluster representative.



11

11

K-Means vs k-Medoids

Scenario	Use K-Means?	Use K-Medoids?
Large datasets	✓ Yes	✗ No (Slow)
Data with outliers	✗ No (Sensitive)	✓ Yes (Robust)
Spherical clusters	✓ Yes	✓ Yes
Arbitrary-shaped clusters	✗ No	✓ Yes
Fast computation needed	✓ Yes	✗ No



Dr. Ali Hassan, DC&SE, College of E&ME, NUST

12

12

MEAN SHIFT CLUSTERING



13

13

Mean Shift

Mean Shift doesn't fix the number of clusters beforehand.

It tries to find the **modes (peaks)** of this distribution.

Think of it as **climbing uphill** to the nearest **peak of data density**.



14

14

Algorithm



For each data point, define a **window (bandwidth radius)**.



Compute the **mean of all points inside that window**.



Shift the window's center to this mean — this is the “mean shift.”



Repeat until convergence (the window stops moving).



Points whose windows converge to the same location are assigned to the **same cluster**.



Dr. Ali Hassan, DC&SE, College of E&ME, NUST

15

15

Example

1D Data points (x):

[1, 2, 3, 8, 9, 10]

Suppose window is 3



16

16

Algorithm: Step 1

We'll start with the first point, $x = 1$.

Step 1: Window around 1

Window range = $[1 - 3, 1 + 3] \rightarrow [-2, 4]$

Points in this window = $[1, 2, 3]$

Mean = $(1 + 2 + 3) / 3 = 2.0$

👉 Shift window center from 1 $\rightarrow$ 2.0



17

17

Algorithm: Step 2

Step 2: Window around new center (2.0)

New window = $[2 - 3, 2 + 3] \rightarrow [-1, 5]$

Points inside: $[1, 2, 3]$

Mean = $(1 + 2 + 3) / 3 = 2.0$

✅ Converged at 2.0 $\rightarrow$ Cluster 1 center ≈ 2



18

18

Algorithm: Step 3

Now let's pick another point, $x = 8$.

Step 3: Window around 8

Window range = [5, 11]

Points in window = [8, 9, 10]

Mean = $(8 + 9 + 10) / 3 = 9.0$

Shift center from 8 $\rightarrow$ 9.0

1D Data points (x):

[1, 2, 3, 8, 9, 10]

Suppose window is 3



19

19

Algorithm: Step 4

Step 4: Window around new center (9.0)

Window range = [6, 12]

Points in window = [8, 9, 10]

Mean = 9.0

✓ Converged at 9.0 $\rightarrow$ Cluster 2 center ≈ 9



20

20

Result

Cluster	Members	Center
1	[1, 2, 3]	2
2	[8, 9, 10]	9



21

21

Keypoints

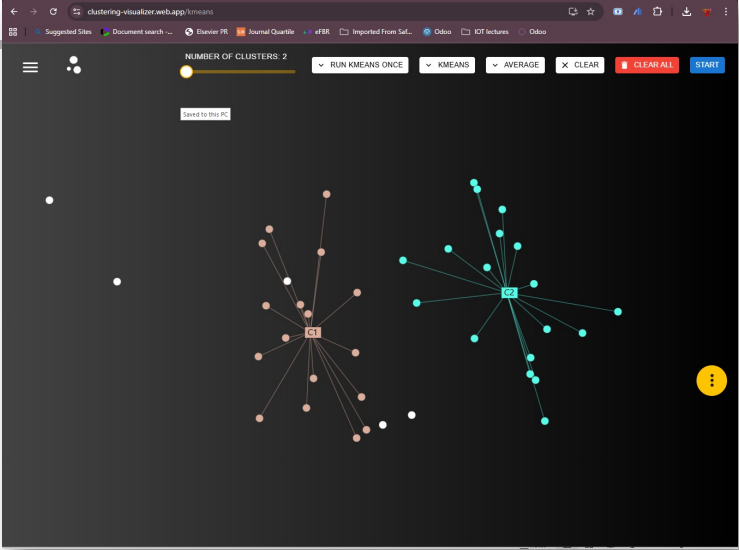
- No need to specify K.
- Finds **arbitrary-shaped clusters** (not just circular).
- Depends on **bandwidth** — the window size controlling cluster granularity.
- Computationally heavier than K-Means.



22

22

Demo



<https://clustering-visualizer.web.app/mean-shift>



Dr. Ali Hassan, DC&SE, College of E&ME, NUST

23

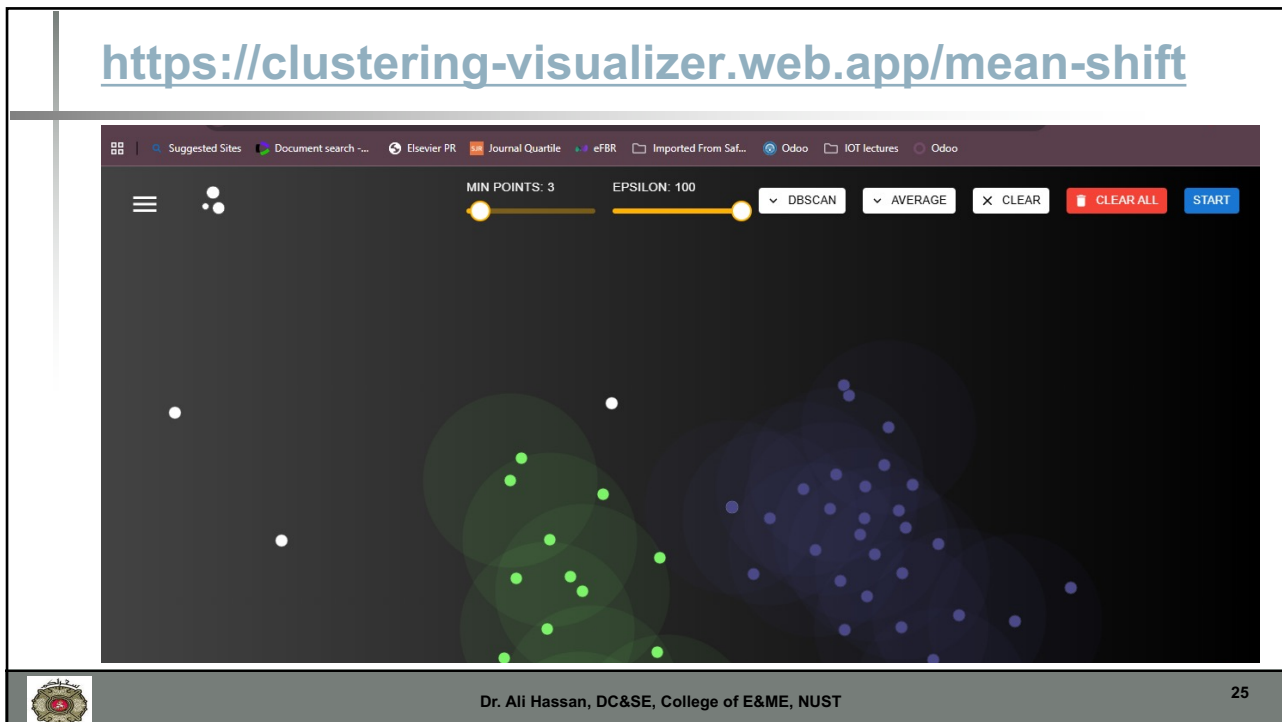
DENSITY BASED CLUSTERING

DBSCAN



Dr. Ali Hassan, DC&SE, College of E&ME, NUST

24



25

DBSCAN

- Density based clustering algorithms divides your entire dataset into **dense regions** separated by sparse regions.
- Unlike K-Means:
 - You **don't need to specify** the number of clusters.
 - It can find **arbitrary-shaped clusters** (not just circles or spheres).
 - It can **ignore noise**.
- DBSCAN expands clusters **based on density** rather than distance to a centroid.

Dr. Ali Hassan, DC&SE, College of E&ME, NUST

26

Density Based Clustering (DBSCAN)

- k-means - need to know k and may cluster unrelated points
- DBSCAN - cluster “dense” regions into clusters

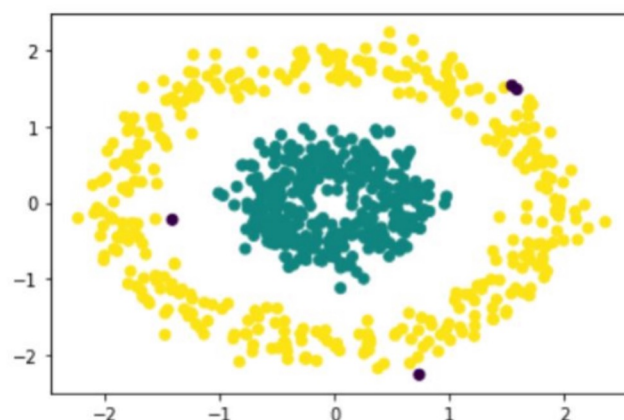


Dr. Ali Hassan, DC&SE, College of E&ME, NUST

27

27

DBSCAN vs. k-means



- clusters reflect distribution of objects

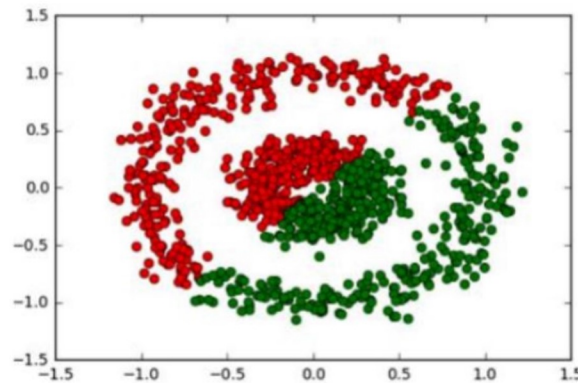


Dr. Ali Hassan, DC&SE, College of E&ME, NUST

28

28

DBSCAN vs. k-means



- k-means does not reflect distribution of objects



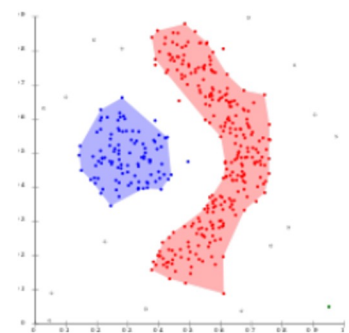
Dr. Ali Hassan, DC&SE, College of E&ME, NUST

29

29

Density-Based Clustering

- Basic Idea
 - Clusters are dense regions in the data space, separated by regions of lower density
 - A cluster is defined as a maximal set of density-connected points
 - Detect arbitrarily shaped clusters Method
- DBSCAN (Density-Based Spatial Clustering of Applications with Noise)



Dr. Ali Hassan, DC&SE, College of E&ME, NUST

30

30

Parameters

Parameter	Meaning
<code>eps (ϵ)</code>	Maximum distance between two points to be considered neighbours
<code>min_samples</code>	Minimum number of points required to form a dense region.



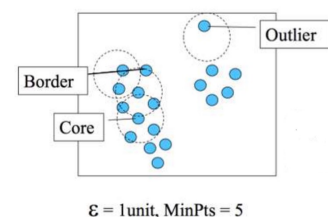
Dr. Ali Hassan, DC&SE, College of E&ME, NUST

31

31

Classifications of points in DBSCAN

- Core point:**
A point that has **at least `min_samples`** points within its **`eps`** radius.
→ Forms the "heart" of a cluster.
- Border point:**
A point that is **within `eps` of a core point**, but **does not have enough neighbors** to be a core point itself.
→ It's part of the cluster but lies on the edge.
- Noise point (outlier):**
A point that is **not within `eps` of any core point**.
→ It doesn't belong to any cluster — it's labelled **-1** in the output.



<https://clustering-visualizer.web.app/>



32

32

WORKING EXAMPLE



Dr. Ali Hassan, DC&SE, College of E&ME, NUST

33

33

Algorithm: Step 1

Step 1 — Pick any unvisited point

- If it's **not** a core point → mark as **noise** (for now) and continue.
- If it **is** a core point → **start a new cluster**.



34

34

Algorithm: Step 2

Step 2 — Expand the cluster

- Retrieve all points within ϵ of this core point $\rightarrow N_\epsilon(p)$
- Add all these points to the current cluster.

Now, for **each point** q in $N_\epsilon(p)$:

- If q is a **core point**, find its neighbors too, and **add them** to the cluster.
- This expansion continues **recursively** — every reachable core point's neighbors are included



35

35

Algorithm : Step 3

Step 3 — Border points

- Points that are **within ϵ of any core** but are **not cores themselves** are also added to the cluster.
- However, they **do not expand** the cluster further.



36

36

Algorithm: Step 4

Step 4 — Outliers

- Points that are never included in any cluster remain labelled as **noise/outliers**.



37

37

Example

Point	x	y
P ₁	1	2
P ₂	2	2
P ₃	2	3
P ₄	8	7
P ₅	8	8
P ₆	25	80



38

38

Calculate distances

Assume $\text{eps} = 4$, $\text{min samples} = 2$

P_1 : within 4 units $\rightarrow P_2, P_3 \rightarrow$ ✓ core point

P_2 : within 4 units $\rightarrow P_1, P_3 \rightarrow$ ✓ core point

P_3 : within 4 units $\rightarrow P_1, P_2 \rightarrow$ ✓ core point

P_4 : within 4 units $\rightarrow P_5 \rightarrow$ ✓ core point

P_5 : within 4 units $\rightarrow P_4 \rightarrow$ ✓ core point

P_6 : no close neighbors $\rightarrow$ ✗ noise

It keeps a queue of points, and checks its neighbours one by one



39

39

Clusters formed

Cluster 1 $\rightarrow \{P_1, P_2, P_3\}$

Cluster 2 $\rightarrow \{P_4, P_5\}$

Noise $\rightarrow \{P_6\}$



40

40

Comparison

Feature	K-Means	DBSCAN
Requires <i>k</i>	✓ Yes	✗ No
Handles noise	✗ No	✓ Yes
Cluster shape	Usually spherical	Arbitrary
Sensitivity to outliers	High	Low
Works with density?	✗ No	✓ Yes



41

41

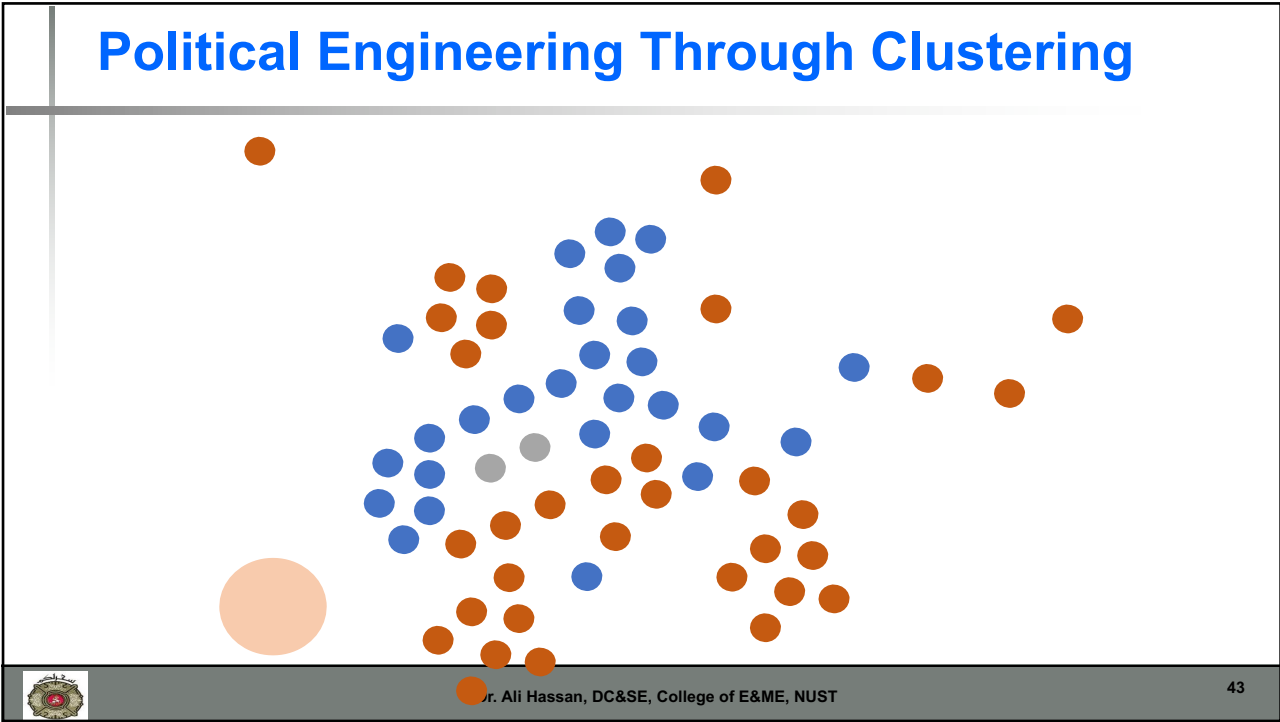
A Working Example



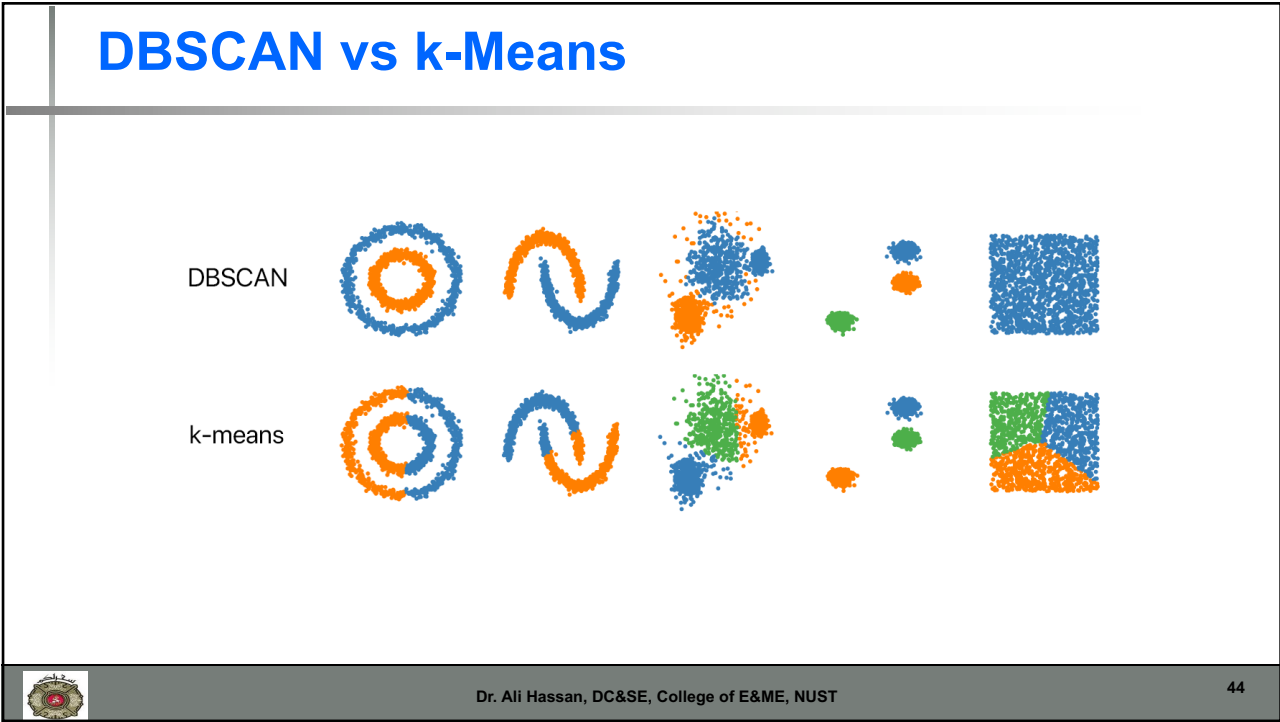


Dr. Ali Hassan, DC&SE, College of E&ME, NUST42

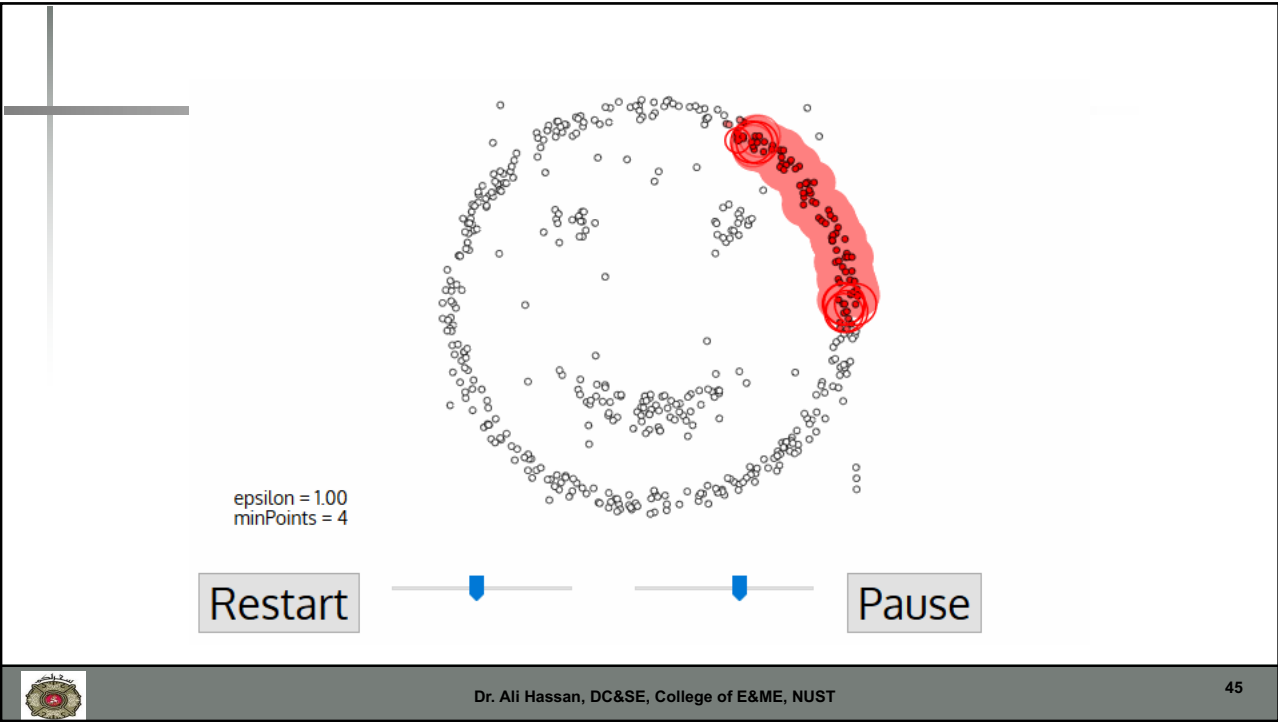
42



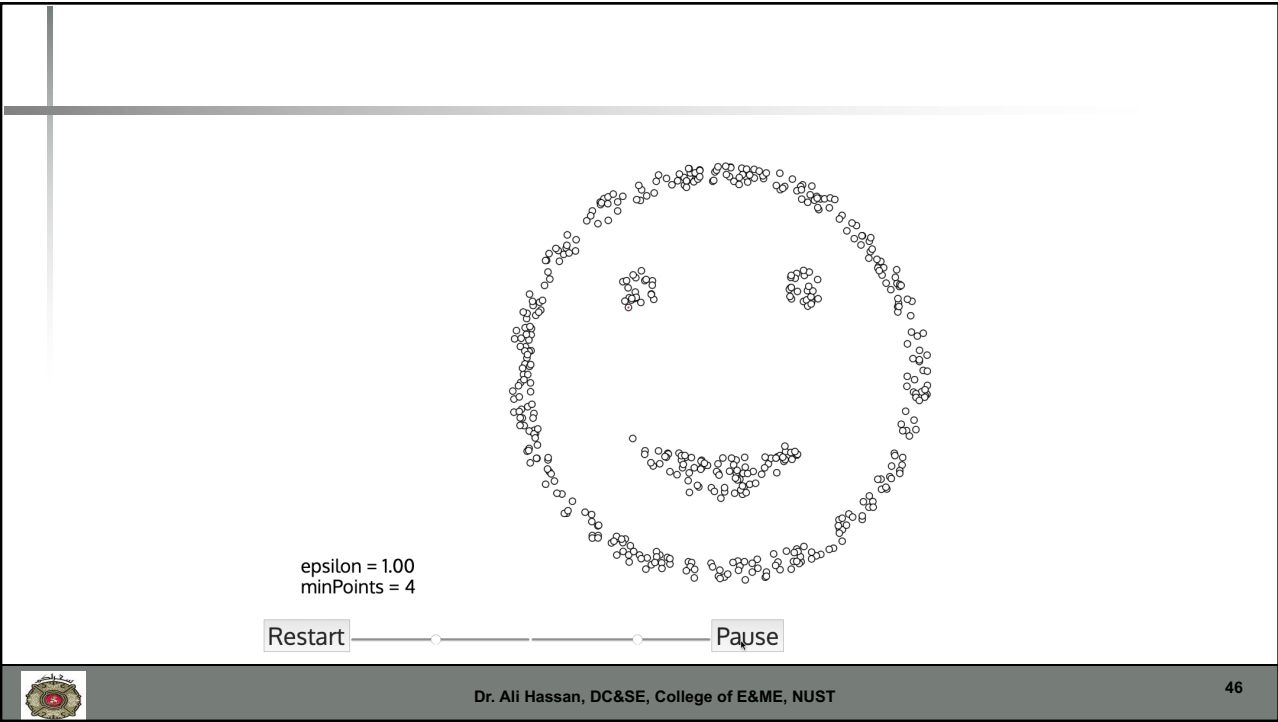
43



44

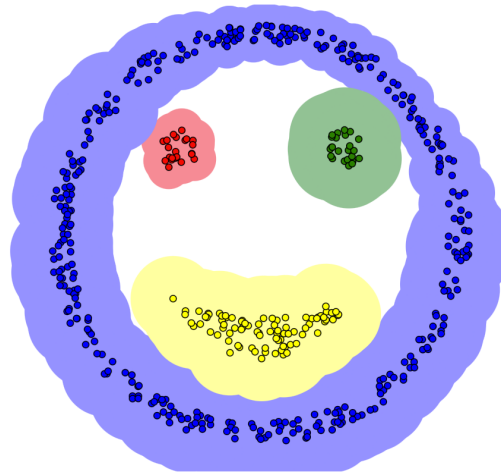


45



46

<https://www.naftaliharris.com/blog/visualizing-dbscan-clustering/>



epsilon = 1.98
minPoints = 4

Restart



Dr. Ali Hassan, DC&SE, College of E&ME, NUST

47

47

DBSCAN in Python

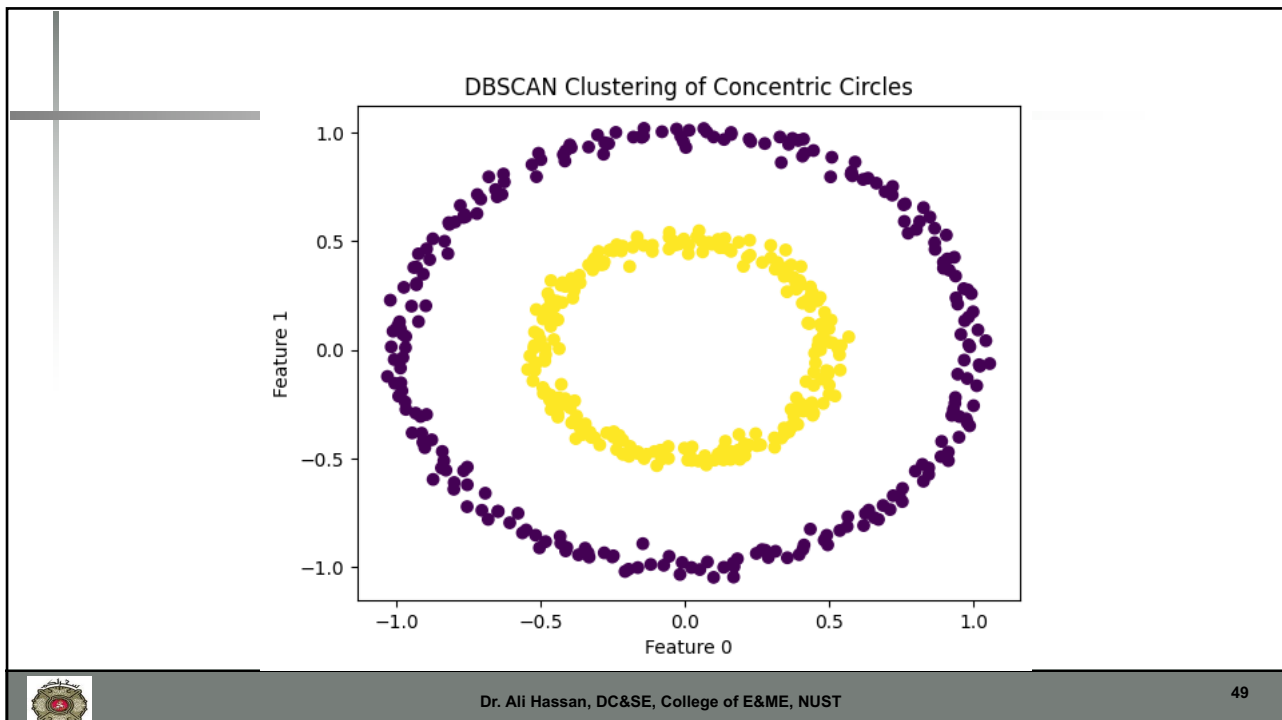
```
import matplotlib.pyplot as plt
from sklearn.datasets import make_circles
from sklearn.cluster import DBSCAN
import numpy as np
# Create a concentric circle dataset
X, _ = make_circles(n_samples=500, factor=.5, noise=.03, random_state=4)
# Apply DBSCAN to the dataset
dbscan = DBSCAN(eps=0.1, min_samples=5)
clusters = dbscan.fit_predict(X)
# Plotting
plt.scatter(X[:, 0], X[:, 1], c=clusters, cmap='viridis', marker='o')
plt.title("DBSCAN Clustering of Concentric Circles")
plt.xlabel("Feature 0")
plt.ylabel("Feature 1")
plt.show()
```



Dr. Ali Hassan, DC&SE, College of E&ME, NUST

48

48



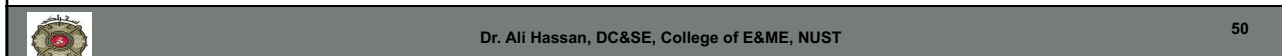
49

Kmeans on same dataset

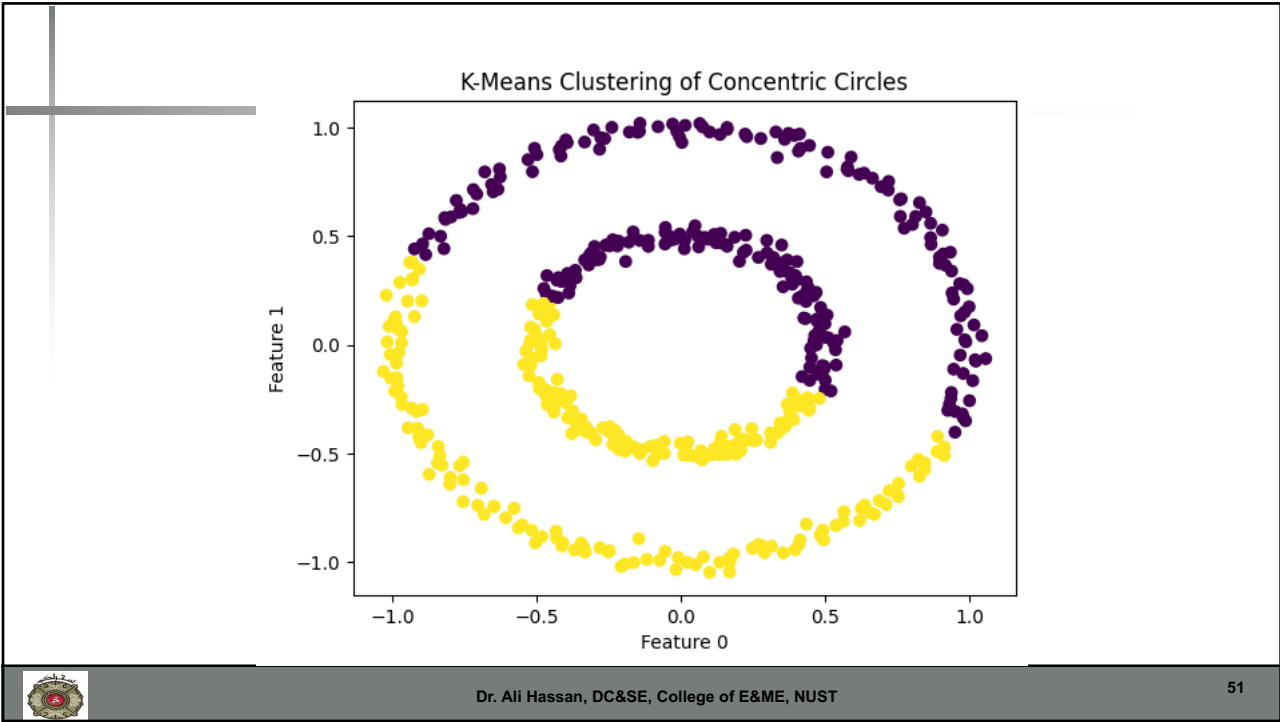
```
from sklearn.cluster import KMeans

# Assuming X is your dataset from make_circles
kmeans = KMeans(n_clusters=2, random_state=0) # You can adjust
n_clusters based on your needs
kmeans_clusters = kmeans.fit_predict(X)

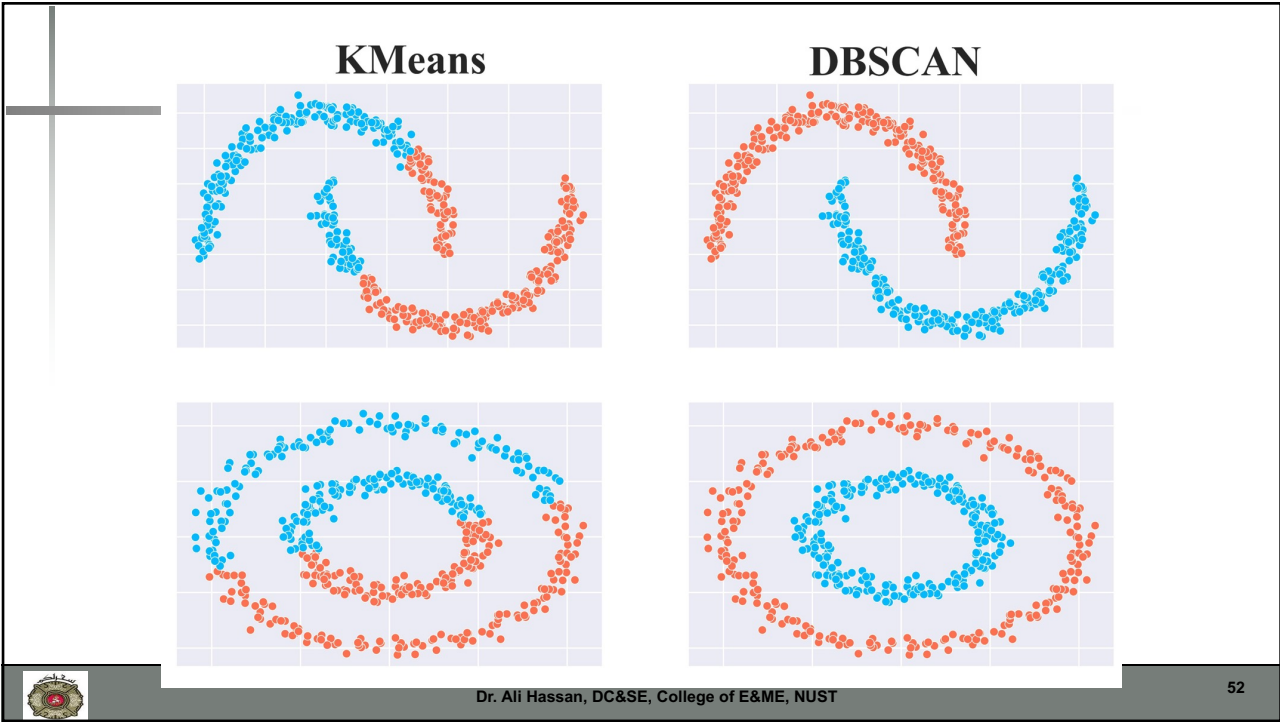
# Plotting
plt.scatter(X[:, 0], X[:, 1], c=kmeans_clusters, cmap='viridis',
            marker='o')
plt.title("K-Means Clustering of Concentric Circles")
plt.xlabel("Feature 0")
plt.ylabel("Feature 1")
plt.show()
```



50



51



52

Ordering Points To Identify Clustering Structure

OPTICS



53

53

Intro

- OPTICS (Ordering Points To Identify Clustering Structure)
- OPTICS is an **extension of DBSCAN** that removes the need to pick a single ϵ value.
- It produces an **ordering of points** and a **reachability plot** to show cluster structure across **multiple density levels**.



54

54

Reachability distance

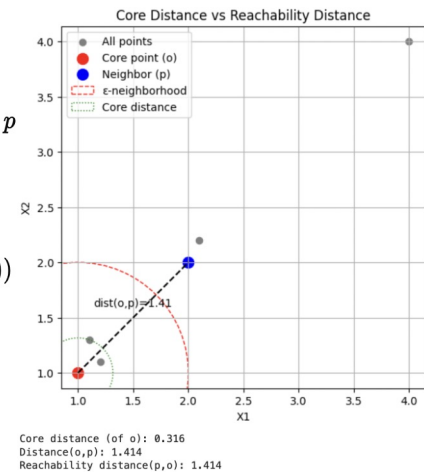
Core distance — the minimum radius ϵ required to make a point a core point.

$$\text{core_dist}(p) = \text{distance to minPts-th nearest neighbor of } p$$

(undefined if p is not a core point)

Reachability distance — how close one point can be *reached* from a core point.

$$\text{reachability_dist}(p, o) = \max(\text{core_dist}(o), \text{distance}(o, p))$$



55

55

Example

Points = [1, 2, 3, 10, 11, 12, 30]

minPts = 3

ϵ (max) = ∞ (just to find all neighbors)



56

56

Distances

◆ Step 1: Compute pairwise distances

From / To	1	2	3	10	11	12	30
1	0	1	2	9	10	11	29
2	1	0	1	8	9	10	28
3	2	1	0	7	8	9	27
10	9	8	7	0	1	2	20
11	10	9	8	1	0	1	19
12	11	10	9	2	1	0	18
30	29	28	27	20	19	18	0



57

57

Iteration 1: Core distances

Point	Distances Sorted	3rd Smallest	Core Distance
P1	[0, 1, 2, 9, 10, 11, 29]	2	2
P2	[0, 1, 1, 8, 9, 10, 28]	1	1
P3	[0, 1, 2, 7, 8, 9, 27]	2	2
P4	[0, 1, 2, 7, 8, 9, 20]	2	2
P5	[0, 1, 1, 8, 9, 10, 19]	1	1
P6	[0, 1, 2, 9, 10, 11, 18]	2	2
P7	[0, 18, 19, 20, 27, 28, 29]	19	19



58

58

Reachability based on point 1 (Core Dist=2)

$$\text{new reachability}(o) = \max(\text{core_dist}(P1), \text{distance}(P1, o))$$

To	dist(P1, o)	max(core_dist(1), dist)	Update
P2	1	2	reach(P2) = 2
P3	2	2	reach(P3) = 2
P4	9	9	reach(P4) = 9
P5	10	10	reach(P5) = 10
P6	11	11	reach(P6) = 11
P7	29	29	reach(P7) = 29

Update order list: [P1]



59

59

Update reachability based on Point 2 (Core Dist=1)

$$\text{new reachability}(o) = \max(1, \text{distance}(P2, o))$$

If smaller than current, update.

To	dist(P2, o)	old reach	new (max)	min(old,new)
P3	1	2	1	1 (updated)
P4	8	9	8	8 (updated)
P5	9	10	9	9 (updated)
P6	10	11	10	10 (updated)
P7	28	29	28	28 (updated)

Update order list: [P1, P2]



60

60

Update reachability for Point 3

To	$\text{dist}(P3, o)$	old reach	new	$\text{min}(\text{old}, \text{new})$
P4	7	8	7	7 (updated)
P5	8	9	8	8 (updated)
P6	9	10	9	9 (updated)
P7	27	28	27	27 (updated)

Update order list: [P1, P2, P3]



61

61

Update reachability for Point 4

To	$\text{dist}(P4, o)$	old reach	new	min
P5	1	8	2	2 (updated)
P6	2	9	2	2 (updated)
P7	20	27	20	20 (updated)

Update order list: [P1, P2, P3, P4]



62

62

Update reachability for Point 5

To	dist(P5, o)	old	new=max(1,dist)	min
P6	1	2	1	1 (updated)
P7	19	20	19	19 (updated)

Update order list: [P1, P2, P3, P4, P5]

63

63

Update list

Update order list: [P1, P2, P3, P4, P5, P6]

Update order list: [P1, P2, P3, P4, P5, P6, P7]

64

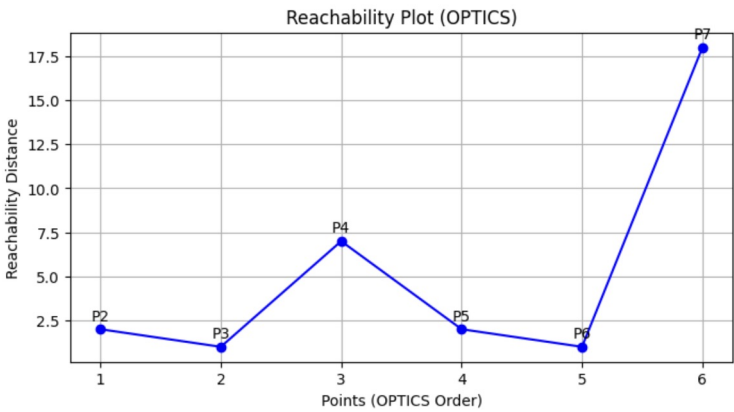
64

Final Reachability Table

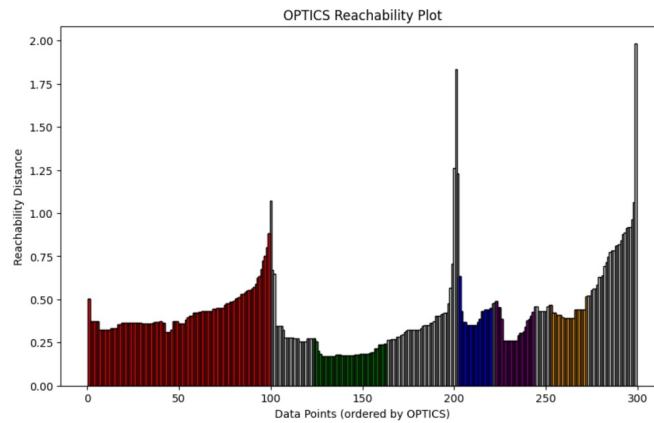
Order	Point	Reachability Distance
1	1	Undefined (start)
2	2	2
3	3	2
4	10	7 (jump)
5	11	2
6	12	2
7	30	∞



Reachability graph



Reachability Graph: Sample



67

67

Final clusters

From the reachability plot:

- Cluster 1: {1, 2, 3} (low reachability values near 1–2)
- Cluster 2: {10, 11, 12} (reachability again low)
- Noise / distant: {30} (large reachability 18)



68

68

THANKYOU



69

69